

EJERCICIO 2 DE REPASO DEL TEMA 1

Implementa la clase Fraccion para que el main() pueda ejecutarse y genere la salida indicada.

```
int main() {
    Fraccion a, b(5);
    const Fraccion c(7,4), d(b), e(5,-4);
    cout << "El numerador de c vale " << c.getn() << endl;
    if (d>a)
        a=-c;
    b+=e;
    cout << a << endl << b << endl << c << endl << d << endl << e << endl;
    cout << "el entero mas cercano a la fraccion " << c << " es " << (int)c << endl;
    cout << "el entero mas cercano a la fraccion " << e << " es " << (int)e << endl;
    cout << c << " = " << (float)c << endl;
    cout << e << " = " << (float)e << endl;
    system("PAUSE");
}
```

Salida:

```
El numerador de c vale 7
-7/4
15/4
7/4
5/1
-5/4
el entero mas cercano a la fraccion 7/4 es 2
el entero mas cercano a la fraccion -5/4 es -1
7/4 = 1.75
-5/4 = -1.25
Presione una tecla para continuar . . .
```

Restricciones:

- a) La clase solo puede tener un único constructor
- b) No puede tener funciones amigas
- c) El operador += debe implementarse mediante una función
- d) Cuando sea posible los operadores deben implementarse mediante métodos

SOLUCION:

Como la clase no tiene memoria dinámica, no hace falta programar un destructor (ya que no hay memoria dinámica que liberar), ni el constructor de copia, ni el operador de asignación, ya que los que genera de oficio el compilador (hacen una copia binaria de los datos) funcionan perfectamente al no haber punteros.

Fraccion.h

```
#ifndef FRACCION_H
#define FRACCION_H

#include <iostream>

using namespace std;

class Fraccion {
    int numerador, denominador;
public:
    Fraccion(int numerador=0, int denominador=1);
    virtual ~Fraccion() { }
    int getn() const { return numerador; } //necesarios para la funcion operator+=
    int getd() const { return denominador; } //necesarios para la funcion operator+=
    void set(int n=0, int d=1); //necesarios para la funcion operator+=
    bool operator>(Fraccion f) const;
    Fraccion operator-() const;
    operator int() const;
    operator float() const { return (float)numerador/denominador; }
};

Fraccion & operator+=(Fraccion &f1, const Fraccion &f2);
ostream& operator<<(ostream &s, const Fraccion &f);

#endif // FRACCION_H
```

Fraccion.cpp

```
#include "Fraccion.h"

Fraccion::Fraccion(int numerador, int denominador) {
    this->numerador=numerador;
    this->denominador=denominador;
}

void Fraccion::set(int n, int d) { numerador=n; denominador=d; }

bool Fraccion::operator>(Fraccion f) const {
    return (numerador*f.denominador>denominador*f.numerador);
};

Fraccion Fraccion::operator-() const { return Fraccion(-numerador, denominador); }

Fraccion::operator int() const {
    int i=numerador/denominador;
    float f=(float)numerador/denominador;
    if (f-i>=0.5)
        i++;
    return i;
}

Fraccion & operator+=(Fraccion &f1, const Fraccion &f2) {
    f1.set(f1.getn()*f2.getd()+f2.getn()*f1.getd(), f1.getd()*f2.getd());
    return f1;
}

ostream& operator<<(ostream &s, const Fraccion &f) {
    if (f.getd()>0)
        s << f.getn() << "/" << f.getd();
    else //s actúa a modo de cout
        s << -f.getn() << "/" << -f.getd();
    return s;
}
```